Scott Long

Lead Software Engineer · Platform Architecture · Distributed Systems · Applied AI

Minneapolis–St. Paul, MN · longstoryscott@gmail.com

LinkedIn · GitHub · llmmllab

SUMMARY

Lead engineer with 11 years building the foundational systems other teams build on. I own cross-cutting platform surfaces — Terraform module libraries, CI/CD template suites, shared client libraries, identity and access services, API gateway architecture — and I ship them at a velocity that unblocks entire organizations rather than single teams.

Most recently I designed and delivered an **agentic computer-use automation platform** from an empty cloud subscription to production across eleven repositories in four months: an LLM-compiled, self-healing executor that drives browser DOM, Linux desktop GUI, OS, and HTTP surfaces from one engine, plus its control plane, Terraform estate, operator console, and end-to-end test harness. It automates **electronic health record workflows delivered over Citrix HDX** — a class of target widely treated as unautomatable — and repairs its own drifted automations as reviewed pull requests.

I operate deliberately across the whole stack and across team boundaries: Python, C#/.NET, Go, and TypeScript; FastAPI and ASP.NET Core; PostgreSQL, SQL Server, MongoDB, and Redis; Terraform and Azure Container Apps; React micro-frontends. Deep healthcare-domain grounding in EHR integration, FHIR, revenue cycle management, and the compliance posture those demand.

TECHNICAL PROFILE

Languages	Python, C#/.NET, Go, TypeScript/JavaScript, SQL, Java, Bash, PHP, C/C++
Backend	FastAPI, ASP.NET Core, Node.js, Azure Functions, GraphQL, REST, gRPC/protobuf
Frontend	React, TypeScript, Vite, MUI, Piral micro-frontends, Vitest, Playwright
Data	PostgreSQL, SQL Server, MongoDB, Redis, Databricks, JanusGraph/Gremlin, SQLAlchemy, Alembic
Cloud	Azure — Container Apps & Jobs, Service Bus, Event Grid, APIM, Entra ID, Key Vault, Functions, Blob, App Insights; AWS
Infrastructure	Terraform, Docker, Kubernetes, private networking, Workload Identity Federation
CI/CD & Ops	Azure Pipelines, GitVersion, OpenTelemetry, SRE patterns, load testing, observability
Applied AI	LLM code generation & self-healing systems, Anthropic Claude API, prompt contracts & caching, structured outputs, MCP servers, computer-use agents, vision-language grounding, LangGraph, self-hosted vLLM & llama.cpp, multi-GPU inference routing
Embedded	C/C++ on Arduino & ESP32, raw HID, Linux uinput, IMU sensor fusion, k3s on bare metal
Domain	Healthcare revenue cycle management, EHR integration, FHIR/HL7, PHI-handling compliance posture

SELECTED IMPACT

- **5,300+ authored commits across ~50 repositories** (2022–2026) spanning infrastructure, backend services, frontends, and shared libraries — **1,785 in 2026 alone**, the majority on net-new platform architecture.
- **Built an agentic automation platform end to end in four months** — 11 repositories, ~1,030 commits, **+241,000 / −101,000 lines** — from empty cloud subscription to production workload.
- **Migrated a production platform from SQL Server to PostgreSQL** across development, UAT, and production behind a feature flag, then fully decommissioned SQL Server.
- **Authored the shared Terraform module and CI/CD template libraries** that dozens of engineering teams provision and deploy against.
- **Automated a GUI-only enterprise application** with no DOM and no API, using a layered template → OCR → vision-model grounding stack.
- **Ship open source**: four published packages across PyPI and npm (**80+ cumulative releases**) and a self-hosted multi-node LLM serving platform built in the open across six repositories.

PROFESSIONAL EXPERIENCE

Lead Software Engineer — Ensemble Health Partners

June 2022 – Present · Minneapolis, MN

Healthcare revenue cycle management. Progressed from delivering platform services to owning the foundational, cross-cutting capabilities that dozens of engineering teams depend on daily — consulted across the org from junior engineers to principal architects.

Agentic computer-use automation platform (2026)

Sole architect and primary engineer of a greenfield platform replacing brittle commercial RPA with deterministic, self-repairing automation.

- Designed and built a **self-healing computer-use executor**: an LLM compiles declarative workflow specs *once* into committed, deterministic replay scripts; at runtime, when a UI target drifts, the engine repairs itself and emits the fix. This keeps the model **off the hot path** entirely, so steady-state execution is plain browser/desktop automation — fast, cheap, auditable — while still absorbing UI change automatically.
- Architected a **three-tier healing model** — element re-location, single-step re-plan, and whole-workflow re-plan — unified across **four automation surfaces** (browser DOM, Linux desktop GUI, OS/shell, HTTP) behind one registry-driven abstraction, with per-surface prompt contracts versioned as reviewable artifacts rather than buried in code.
- Made every repair **land as a reviewed pull request**, so automation improves under version control with a human in the loop instead of silently mutating in production — turning self-healing from an operational risk into a code-review workflow.
- Solved desktop automation where **no DOM exists** by building a layered grounding stack — deterministic template matching → OCR → vision-language model — so the expensive vision path runs only as a last resort. Deployed and benchmarked a **self-hosted vision-language grounding endpoint at 84% accuracy** behind a pluggable backend protocol, on private GPU compute with its own model-staging pipeline for a network-isolated registry.
- **Automated a GUI-delivered EHR application over Citrix HDX** by refusing a bespoke integration: composed the remote-session handoff out of atomic primitives already available on the existing surfaces, then drove the full clinical lookup and coverage-update workflows end to end **without any vision-model dependency at steady state**.
- Built the **control plane** (FastAPI, PostgreSQL, Alembic, Azure Service Bus, Entra ID JWT auth, OpenTelemetry): run lifecycle state machine, per-tenant work queues with **serial-in-order session guarantees** so one tenant never has two batches in flight while hundreds run in parallel, stuck-run reaper, force-stop, claim/release semantics, and outcome ledgers derived from raw run facts.
- Executed the **SQL Server → PostgreSQL migration** across three environments: translated T-SQL stored procedures to Postgres dialect, introduced Alembic as the migration authority, built an **ephemeral in-network migration runner requiring no standing infrastructure**, cut over development → UAT → production behind a feature flag, then decommissioned SQL Server entirely.
- Owned the **Terraform estate across per-environment cloud subscriptions**: Container Apps Jobs, private ingress via private endpoints, egress routed through the existing corporate network path rather than bespoke infrastructure, messaging with public network access closed, Workload Identity Federation service connections, and a bootstrap chain owning the **entire path from an empty subscription to `apply`**.
- **Collapsed a fragmenting per-surface architecture into one image, one queue, one job** — eliminating a class of drift where a rehearsal and a production run could silently differ, and removing any need for upstream systems to know which surface a workflow used. Consolidated four container images and two registries into one.
- Built an **MCP (Model Context Protocol) authoring toolchain** that lets an engineer rehearse a workflow against the live application inside a watchable container, then generate the spec and replay script *from what was actually observed* — collapsing the entire surface to a single dispatch tool so one permission rule covers a full authoring session, with policy that prose could not enforce moved into deterministic pre-execution hooks.
- Shipped the surrounding platform: an **operator console** (React, TypeScript, Vite, MUI, Vitest, Playwright), a containerized **end-to-end harness** gating CI, **task ingestion** and **egress** services, and a **credentials broker** that mints TOTP/MFA codes at replay time so no secret is ever written into a generated script or a log.
- Held a **90% coverage gate** across the engine with full dependency injection — no real browser, screen, network, or cloud dependency in the unit suite.

Enterprise identity & access management (2023–2025)

- Led the access-management program **end to end through team turnover and shifting scope**, delivering the .NET service suite that became the foundation for the organization's identity-governance program: a centralized **directory mirror service** exposing consistent role-aware identity to every platform service, an **event-driven synchronization service** reconciling directory state over message bus and event grid, a **file ingestion and validation service** with configurable business rules and downstream fan-out, and a **shared library and package suite** consumed across the platform.
- **Made directory group-membership synchronization dramatically more efficient** by restructuring event handling and group add/remove operations, and eliminated production race conditions in user synchronization with Redis-backed caching, queued-task telemetry, and idempotent upserts.
- Retired substantial technical debt and **closed standing security gaps** by consolidating fragmented, ad-hoc access paths behind a single auditable service boundary, with an auditable base contract and asynchronous audit logging throughout.
- Added load testing to the CI/CD pipeline and drove the telemetry that made a long-standing production-only synchronization defect diagnosable.

Foundational platform & shared capabilities (2022–2026)

- Author and owner of the organization's **shared CI/CD pipeline template library** — build, test, lint, semantic versioning, IaC deployment, and load-test stages — fully parameterized for multi-environment promotion and multi-agent-pool routing, and used as the default delivery path across the org.
- Built the **shared Terraform module library** abstracting complex cloud architecture into single-call capabilities, and defined the **security domains**, resource-group topology, and access-control patterns that dozens of teams provision against — architectural ownership of shared plumbing on behalf of the whole engineering organization.
- Designed and delivered a **FHIR API backend** for electronic health record integration, enabling partner programs to go live against the broader healthcare ecosystem.
- Delivered a **contingent-worker provisioning integration with Workday**, tying together HR, identity, security, and engineering systems and materially accelerating onboarding approvals — cited in performance review as a quality, scalable solution spanning four organizations.
- Established the **micro-frontend architecture** (Piral + React) allowing every team to contribute to one shell application, and contributed to the shared **design-system theme package** consumed org-wide.
- Built a suite of **internal data-platform services**: an audit-logging service (Azure Functions; GraphQL for flexible reads, REST for simple writes, MongoDB-backed, with JSON-Schema-governed log contracts and a schema standard adopted org-wide), a data-pipeline mapping service, a client-metadata service establishing one source of truth, and an automated notification service for the data-engineering organization.
- Created the roadmap, **API gateway architecture (Azure APIM)**, and production deployment process for the **enterprise application integration platform**, presenting architecture to cross-functional stakeholders to align technical direction and operational readiness.
- Delivered a complete **clinical advisory product in a four-week skunkworks** — FastAPI + async SQLAlchemy backend, React/Vite frontend, data model, and Terraform estate — across five repositories.
- Championed operational readiness org-wide: intraday monitoring, recovering caches, telemetry automation across teams, and the runbooks that make them supportable.

Leadership & influence

- Ran interview loops and onboarding; consulted daily with engineers from junior through lead-and-above on integration patterns, platform capabilities, and design trade-offs — coaching by asking the right questions rather than prescribing answers.
- Authored the durable written record: architecture diagrams, runbooks, module documentation, onboarding and operational guides, and how-to recipes on the shared technical docs site — so institutional knowledge outlives any individual contributor.
- Recognized in performance reviews for architecting across multiple services and teams, keeping complex multi-discipline initiatives on schedule through organizational change, and **driving tense discussions toward objective, fact-based decisions**.
- Modeled the learning culture as a participant in internal GenAI upskilling programs and cross-team architecture refinement sessions.

Senior Software Engineer — Optum (UnitedHealth Group)

August 2020 – June 2022 · Minneapolis, MN

Led the platform team for a big-data claims ingestion and self-service pipeline platform.

- Led the platform team responsible for **self-service data pipeline configuration and ingestion**, removing manual provisioning overhead for every downstream consuming team.
- Rebuilt a historical claims loader to hit a hard **six-week deadline for 100,000+ data files** (~10 GB each): replaced per-file temp-directory staging with **end-to-end transformation streams** piping compressed archives directly into the validation engine for a **~4× throughput gain**, then added a size-aware adaptive upload-pacing algorithm for a **further ~2×**. Delivered the full backlog inside the deadline.
- Wrote a CLI tool that **automated an entirely manual database schema-configuration process in three days** and trained the team on it — **saving an estimated 1,000+ engineering hours over the following six months**. Self-initiated, outside assigned scope.
- Built a **MongoDB single source of truth for client metadata**, collapsing duplicated and conflicting records across systems.
- Introduced **GraphQL and JSON-Schema validation** to the platform, **defined the org's schema standard, structure, and supporting APIs**, and delivered the **OAuth2 service-to-service authentication strategy**.
- Solved a hard infrastructure constraint by resolving customer identifiers at Terraform apply time through a provisioner reading a network-private API otherwise unreachable from the cloud — **zero missing-identifier incidents since**.
- Established TDD patterns for code contribution, maintained the Terraform-managed cloud estate, and won a policy change loosening single-code-owner PR approval that had been the team's primary delivery bottleneck — after which the team **stopped missing deadlines** and began regular demos.

Software Engineer — Bluespire Marketing

August 2019 – August 2020 · Minneapolis, MN

- Built complex data-driven applications in **C# (.NET / .NET Core)** and designed the supporting SQL Server schema and stored-procedure layer.
- Created the team's **DevOps pipelines and automated deployments**, replacing manual release steps, and brought a costly offshore support contract back in-house.
- Integrated eCommerce payment-gateway flows in JavaScript.

Earlier Experience

Web Developer — Scientific Societies *(Dec 2017 – Aug 2019)* — Extended SharePoint with custom C#/.NET functionality. Wrote a modern JavaScript library **backward-compatible to IE6** — a hard constraint of the legacy SharePoint version — and open-sourced it for continued use after departure. Built Python/Node.js file-manipulation and web-crawling tooling, including a broken-link crawler for a large raw-HTML site.

Freelance Developer *(Jan 2016 – Present)* — Linux systems administration (CentOS/Ubuntu) and full-stack delivery for small businesses: React frontends paired with WordPress admin interfaces, and REST APIs in Node.js.

Web Developer — Virtus Law *(Jun 2015 – May 2017)* — Built software to collect client data and **generate legal documents automatically**, removing the bulk of manual document preparation. Redesigned and maintained the public and internal sites.

PERSONAL & OPEN-SOURCE ENGINEERING

The same systems work, done in the open. github.com/longstoryscott · github.com/llmmllab

- **[llmmllab](https://github.com/llmmllab)** — a self-hosted, multi-node **LLM inference, evaluation, and serving platform** built across six repositories, with **34 merged pull requests** of my own. A FastAPI inference service exposing **OpenAI-, Anthropic-, and Ollama-compatible endpoints**; a llama.cpp server manager and request proxy performing **VRAM-pressure-aware eviction that is tensor-split aware** and routing requests to warm free-slot peers rather than VRAM-tight ones; **LangGraph** agent orchestration; gRPC/protobuf service contracts shared as a submodule; a React UI; and k3s deployment manifests. Production- shaped distributed GPU scheduling, at homelab scale, on my own hardware.
- **[schema2code](https://pypi.org/project/schema2code/)** *(PyPI)* — generates typed models from JSON/YAML Schema across **five targets**: Go, Python (Pydantic or dataclasses), TypeScript, C#/.NET, and protobuf. Its current direction lets **LLMs call tools by writing sandboxed Python instead of JSON tool calls — measured at 70% fewer tokens and 66% fewer round trips.**
- **MCP servers** — `mcp-server-gmail` and `mcp-server-web`: **FastMCP** servers with **OAuth 2.0 via Dex + OpenLDAP**, dual stdio/HTTP transport for both local and networked clients, and Kubernetes deployment.
- **[k3s-cluster](https://github.com/longstoryscott/k3s-cluster)** — bare-metal **Raspberry Pi Kubernetes** cluster from scratch: k3s across nodes, a private container registry, nginx ingress, and Gateway API — the substrate the llmmllab services actually run on.
- **Published npm packages** — **react-gallery-designer** (v1.4.5, 34 releases), **react-image-designer** (v1.2.1, 25 releases, progressive image loading), and **long-story-library** (23 releases); plus **LongStoryPress**, a headless WordPress multisite behind nginx serving an SSR React frontend.
- **[magic-mouse-gnome](https://github.com/longstoryscott/magic-mouse-gnome)** — reads **raw HID touch data** off a Magic Mouse 2 to synthesize swipe gestures on Linux/Wayland, working around GNOME's missing virtual-keyboard protocol by driving the kernel **uinput** framework directly.
- **Embedded & robotics** *(C/C++, Go, TypeScript)* — an Arduino **quadcopter flight controller** and paired RC controller with IMU sensor fusion, a mecanum-wheel rover, a Raspberry Pi + Arduino wireless IoT/robotics controller, a **full-stack IoT heat pump system** spanning firmware, a Go API, and a TypeScript UI, and a 3D-printer filament-extruder auger controller.
- **Upstream contributions** — `openclaw/openclaw` (per-agent and per-cron-job provider request headers), `abetlen/llama-cpp-python`, and `watzon/wsl-proxy`.

EDUCATION & CERTIFICATIONS

Bachelor of Individualized Studies — Global Studies, Communication Studies, English *University of Minnesota – Twin Cities* · 2006 – 2012 · Minors: Spanish, Psychology Study abroad: Language and Culture, Buenos Aires, Argentina (2011)

Certifications — Microsoft Azure Fundamentals (2023) · OWASP API Security Top 10 (2023) · AI Product Security · RAG with LlamaIndex · Prompt Engineering with LangChain

LANGUAGES

English (native) · Spanish (full professional proficiency) · French (limited working proficiency)